RELATÓRIO TÉCNICO DE AUDITORIA

SIGE — Sistema de Gestão de Estoque

Análise estática do projeto sige-laravel-vue.zip | 11/11/2026 | Versão 1.0

Documento elaborado com base no modelo Relatorio_Auditoria_SES.docx. Não foram reproduzidas credenciais ou informações sensíveis.

1. Identificação e escopo

Sistema	SIGE — Sistema de Gestão de Estoque
Equipe responsável	A preencher pelos integrantes do grupo; não identificada com segurança no pacote
Objeto	Arquivos do ZIP enviado: backend Laravel, frontend Vue, migrations, testes, Docker e documentação
Tecnologias	PHP ^8.2; Laravel ^12; Sanctum ^4; Vue ^3.4; Pinia; Vite; MySQL 8; Nginx; Docker; bibliotecas de PDF, Excel e gráficos
Método	Inventário de arquivos, leitura de código, rotas, configuração, testes e documentação; comparação de regras declaradas
Limitações	Sem inicialização de contêineres, sem banco MySQL operacional, sem navegação em interface e sem testes HTTP end-to-end; testes PHPUnit não executados

2. Resumo executivo

Foram registrados 12 achados sustentados pela inspeção: 4 de gravidade alta, 7 média e 1 baixa. São achados de código, configuração, documentação e riscos; não representam 12 falhas funcionais reproduzidas. A maior prioridade é controlar a distribuição de segredos e revisar a restauração de backup antes de usar dados reais.

Pontos positivos observados: autenticação com Sanctum; limitação explícita de login (throttle:5,1); rotas de escrita com middleware de perfil; módulo de auditoria; testes dedicados a pedidos e recebimentos; validações em fluxos de entrada; estrutura de frontend organizada por páginas e componentes.

3. Metodologia, procedimentos e evidências

• Leitura dos manifestos composer.json e package.json e inventário de 251 arquivos fora de .git, vendor e node_modules.

• Inspeção das rotas da API, dos controllers AuthController e BackupController, do chatbot, do middleware declarado e da persistência de sessão no frontend.

• Comparação de README com requisitos do Composer e dependências do frontend.

• Inspeção do docker-compose.yml, do Nginx, dos arquivos .env/.env.example e dos diretórios de testes.

• Não foi realizada exploração, alteração do banco, execução de migrações nem restauração de backup. Evidências são trechos e caminhos do pacote; capturas de tela da aplicação não estão disponíveis.

4. Achados técnicos detalhados

SIGE-01 — Distribuição de .env e log de execução

Gravidade / prioridade	Alta / P0
Classificação	Confirmado no pacote
Localização	backend/.env; backend/storage/logs/laravel.log
Evidência verificada	O arquivo ZIP contém configuração de ambiente e log de aplicação. A presença de credenciais e chave de aplicação foi verificada sem reproduzir seus valores.
Impacto / ressalva	Exposição de credenciais e de informações operacionais a quem recebe o pacote.
Correção recomendada	Retirar .env e logs dos pacotes; gerar distribuição somente a partir de arquivos versionados; trocar segredos que tenham sido compartilhados.

SIGE-02 — Restauração de backup desabilita integridade referencial

Gravidade / prioridade	Alta / P0
Classificação	Confirmado no código
Localização	backend/app/Http/Controllers/BackupController.php, método restaurarBackup
Evidência verificada	O método executa SET FOREIGN_KEY_CHECKS=0, TRUNCATE em quatro tabelas e INSERT com dados do JSON; a reativação ocorre depois das inserções.
Impacto / ressalva	Possibilidade de inconsistências entre entidades e perda de dados em restaurações incorretas; não foi executada restauração real.
Correção recomendada	Validar integralmente o arquivo e suas relações antes de alterar dados; utilizar banco temporário, estratégia de restauração atômica compatível com MySQL e backup de segurança.

SIGE-03 — Mensagens internas de exceção retornadas ao cliente

Gravidade / prioridade	Alta / P1

Classificação	Confirmado no código
Localização	backend/app/Http/Controllers/BackupController.php, bloco catch
Evidência verificada	Em caso de exceção, a resposta HTTP 500 concatena $e->getMessage() ao texto enviado ao cliente.
Impacto / ressalva	Possível exposição de detalhes do banco e da infraestrutura em erros de restauração.
Correção recomendada	Retornar erro genérico com identificador de incidente e registrar detalhes apenas em log restrito.

SIGE-04 — Ambiente Docker com debug e banco exposto

Gravidade / prioridade	Alta / P1
Classificação	Confirmado na configuração
Localização	docker-compose.yml; backend/.env.example
Evidência verificada	APP_ENV=local e APP_DEBUG=true no backend; MySQL publica 3306:3306 e contém credenciais estáticas no compose.
Impacto / ressalva	Configuração inadequada para produção; o alcance efetivo da porta depende do host e da rede.
Correção recomendada	Criar compose de produção com debug desligado, segredos externos, rede privada para o banco e políticas de acesso.

SIGE-05 — Token de sessão armazenado no localStorage

Gravidade / prioridade	Média / P1
Classificação	Confirmado no código
Localização	frontend/src/servicos/autenticacao.store.js; frontend/src/servicos/api.js
Evidência verificada	O token retornado pelo login é gravado no localStorage e usado nas requisições.
Impacto / ressalva	Em eventual XSS, scripts maliciosos poderiam acessar o token; não foi demonstrada exploração de XSS.
Correção recomendada	Revisar estratégia de autenticação, reforçar CSP e sanitização, reduzir persistência e avaliar cookies HttpOnly em arquitetura compatível.

SIGE-06 — Chave de aplicação preenchida no arquivo de exemplo

Gravidade / prioridade	Média / P1
Classificação	Confirmado no pacote
Localização	backend/.env.example
Evidência verificada	O arquivo de exemplo traz APP_KEY preenchida, além de valores estáticos de banco de desenvolvimento.
Impacto / ressalva	Risco de replicação da mesma chave em novos ambientes e reutilização de senhas de exemplo.
Correção recomendada	Manter APP_KEY vazia no exemplo e documentar php artisan key:generate; não reutilizar senhas.

SIGE-07 — Pré-requisito de PHP divergente

Gravidade / prioridade	Média / P2
Classificação	Confirmado na documentação

Localização	README.md; backend/composer.json
Evidência verificada	O README informa PHP >= 8.1, mas composer.json exige php ^8.2.
Impacto / ressalva	Instalações com PHP 8.1 falharão na resolução de dependências.
Correção recomendada	Corrigir README para PHP >= 8.2 e validar demais pré-requisitos.

SIGE-08 — Instruções de instalação redundantes

Gravidade / prioridade	Média / P2
Classificação	Confirmado na documentação
Localização	README.md; backend/composer.json; frontend/package.json
Evidência verificada	O README manda executar composer require de biblioteca já presente no composer.json e npm install individual de pacotes já declarados no frontend/package.json.
Impacto / ressalva	Aumenta risco de alterações involuntárias nas versões e dificulta reprodução do ambiente.
Correção recomendada	Usar composer install e npm ci com lockfiles íntegros; retirar instalações redundantes.

SIGE-09 — Cobertura de testes limitada frente aos módulos

Gravidade / prioridade	Média / P2
Classificação	Confirmado na inspeção
Localização	backend/tests/Feature; backend/tests/Unit; backend/routes/api.php
Evidência verificada	Foram localizados seis arquivos PHP de teste (incluindo exemplos e TestCase), enquanto a API contempla múltiplos módulos de estoque, pedidos, recebimentos, relatórios, backup, usuários e chatbot.
Impacto / ressalva	Fluxos críticos podem não ter testes automatizados dedicados; não equivale a comprovação de falha funcional.
Correção recomendada	Adicionar testes de autorização, limites, movimentação concorrente, restauração, importação e consistência do estoque.

SIGE-10 — Renderização de HTML no chatbot

Gravidade / prioridade	Média / P2
Classificação	Risco identificado no código
Localização	frontend/src/componentes/ui/ChatbotWidget.vue
Evidência verificada	O componente utiliza v-html para exibir o resultado de renderizarMarkdown; o projeto declara DOMPurify. Não foi comprovada falha de sanitização.
Impacto / ressalva	A segurança depende da sanitização efetiva de todo conteúdo renderizado, inclusive saídas de IA.
Correção recomendada	Cobrir a função com testes de XSS e garantir sanitização imediatamente antes de inserir HTML.

SIGE-11 — Backup exportado não representa toda a aplicação

Gravidade / prioridade	Média / P2
Classificação	Confirmado no código

Localização	backend/app/Http/Controllers/BackupController.php
Evidência verificada	O JSON exportado inclui produtos, lotes, itens de lote e movimentações; não inclui, nesse método, usuários, pedidos, notas de recebimento e logs.
Impacto / ressalva	O nome backup pode sugerir restauração integral, embora o escopo do método seja parcial.
Correção recomendada	Renomear para backup de estoque ou documentar escopo; criar procedimento de backup completo e teste de recuperação.

SIGE-12 — Pastas e projetos paralelos aumentam ambiguidade

Gravidade / prioridade	Baixa / P3
Classificação	Confirmado na estrutura
Localização	backend/; backend_backup/; composer.json e package.json na raiz e em subprojetos
Evidência verificada	O pacote contém backend_backup e manifestos de dependências em múltiplos níveis.
Impacto / ressalva	Pode confundir a instalação e a manutenção se não houver explicação de qual diretório é oficial.
Correção recomendada	Documentar a função de cada pasta e remover cópias obsoletas após validação com a equipe.

5. Segurança e controle de acesso

A API protege o conjunto principal de rotas com auth:sanctum e usa o middleware perfil:admin,operador em operações de escrita. O login possui throttle:5,1 e retorna mensagem genérica para credenciais inválidas. Essas medidas são favoráveis, mas a avaliação de autorização por objeto, revogação de sessões, ataques de força bruta e permissões efetivas requer testes HTTP em ambiente isolado.

Não se afirma que houve SQL Injection ou XSS. As consultas whereRaw identificadas utilizam parâmetros vinculados; o chatbot usa HTML renderizado, cuja sanitização deve ser validada com testes específicos. A presença de .env no ZIP representa risco de distribuição, não prova que esteja versionado no Git.

6. Banco de dados e integridade

Foram localizados 44 arquivos de migration. A aplicação utiliza modelos para produtos, lotes, itens, movimentações, pedidos e recebimentos. O fluxo de recebimento possui testes de proteção contra entrada duplicada, quantidade fracionada e divergência de CNPJ, mas não foi possível executar a suíte. O risco mais relevante identificado na leitura está no procedimento de restauração com desativação de verificações de chaves estrangeiras e truncamento de tabelas.

É necessário executar migrações do zero, verificar FKs, índices e unicidade no MySQL real, testar concorrência de movimentações e simular restauração com dados fictícios. Sem essas etapas, não se pode concluir que a integridade esteja integralmente preservada ou comprometida.

7. Funcionalidades por módulo

Login e perfis	Rotas e middleware examinados; login e restrições não executados via HTTP.
Produtos, lotes e itens	Rotas CRUD e de movimentação identificadas; consistência e interfaces pendentes de execução.
Pedidos e recebimentos	Testes automatizados existentes para pedidos e entrada de notas; resultados não executados nesta auditoria.
Perdas e movimentações	Endpoints presentes; validar saldos, transações e autorização.
Relatórios e exportações	Endpoints CSV/XLSX/PDF identificados; validar conteúdo, fórmulas e volume de dados.
Backup e restauração	Inspeção do código realizada; restauração real deliberadamente não executada.
Chatbot	Rotas com throttle:20,1; revisar segurança do HTML, privacidade e comportamento de fallback.

8. Interface, acessibilidade e desempenho

O frontend usa Vue Router, Pinia e componentes específicos para páginas e layout. Não foram executados testes em navegador. Recomenda-se validar teclado, foco, contraste, leitores de tela, zoom de 200%, telas pequenas, erros de formulário e navegação por perfil. Desempenho e escalabilidade permanecem não avaliados: medir consultas, importações, exportações e acessos simultâneos com dados fictícios.

9. Plano de correção priorizado

P0 — Imediato	SIGE-01 e SIGE-02: remover segredos do material distribuído, avaliar rotação de credenciais e redesenhar a restauração segura.
P1 — Antes de produção	SIGE-03 a SIGE-06: mensagens genéricas, ambiente de produção seguro, revisão de tokens e exemplos sem chave fixa.
P2 — Curto prazo	SIGE-07 a SIGE-11: corrigir README, automatizar testes, validar chatbot e esclarecer escopo do backup.
P3 — Organização	SIGE-12: racionalizar estrutura de diretórios e manifestos.

10. Conclusão técnica

O SIGE apresenta arquitetura coerente para um sistema de estoque, com autenticação, perfis, módulos operacionais e testes de alguns fluxos importantes. Entretanto, a inspeção identifica riscos relevantes na distribuição do pacote, nas

configurações de implantação e no mecanismo de restauração. Não é possível declarar o sistema pronto para produção apenas com a análise estática. Recomenda-se corrigir os itens P0/P1 e realizar homologação funcional e de segurança com evidências reproduzíveis antes de utilizar dados reais.

11. Tabela consolidada

SIGE-01 | Alta | P0 | Distribuição de .env e log de execução

SIGE-02 | Alta | P0 | Restauração de backup desabilita integridade referencial

SIGE-03 | Alta | P1 | Mensagens internas de exceção retornadas ao cliente

SIGE-04 | Alta | P1 | Ambiente Docker com debug e banco exposto

SIGE-05 | Média | P1 | Token de sessão armazenado no localStorage

SIGE-06 | Média | P1 | Chave de aplicação preenchida no arquivo de exemplo

SIGE-07 | Média | P2 | Pré-requisito de PHP divergente

SIGE-08 | Média | P2 | Instruções de instalação redundantes

SIGE-09 | Média | P2 | Cobertura de testes limitada frente aos módulos

SIGE-10 | Média | P2 | Renderização de HTML no chatbot

SIGE-11 | Média | P2 | Backup exportado não representa toda a aplicação

SIGE-12 | Baixa | P3 | Pastas e projetos paralelos aumentam ambiguidade

Anexo A — Evidências e referências de arquivo

As evidências são localizações verificáveis no ZIP recebido. Para cada achado, a equipe deve anexar captura do trecho de código e, quando aplicável, resultado de teste reproduzido em ambiente de homologação, com dados fictícios. Não foram criados prints artificiais nem resultados de execução.

SIGE-01 — backend/.env; backend/storage/logs/laravel.log

SIGE-02 — backend/app/Http/Controllers/BackupController.php, método restaurarBackup

SIGE-03 — backend/app/Http/Controllers/BackupController.php, bloco catch

SIGE-04 — docker-compose.yml; backend/.env.example

SIGE-05 — frontend/src/servicos/autenticacao.store.js; frontend/src/servicos/api.js

SIGE-06 — backend/.env.example

SIGE-07 — README.md; backend/composer.json

SIGE-08 — README.md; backend/composer.json; frontend/package.json

SIGE-09 — backend/tests/Feature; backend/tests/Unit; backend/routes/api.php

SIGE-10 — frontend/src/componentes/ui/ChatbotWidget.vue

SIGE-11 — backend/app/Http/Controllers/BackupController.php

SIGE-12 — backend/; backend_backup/; composer.json e package.json na raiz e em subprojetos

Anexo B — Testes pendentes e critérios de aceite

□ Executar composer install e php artisan test, registrar totais e falhas, sem modificar banco real.

□ Executar npm ci, npm run build e lint, registrando eventuais erros.

□ Subir Docker em ambiente isolado; confirmar portas, acesso ao MySQL e configuração de produção.

□ Criar dados fictícios e testar autenticação, autorização por perfil e por recurso.

□ Testar movimentações simultâneas, recebimento duplicado e saldos após operações.

□ Validar restauração de backup em cópia descartável, verificando integridade antes e depois.

□ Realizar testes de acessibilidade e responsividade no navegador.

□ Realizar análise de dependências com composer audit e npm audit em ambiente com acesso à rede.

12. CHECKLIST DE CORREÇÕES E VALIDAÇÃO FINAL

Este checklist deve ser preenchido pela equipe após a implementação das correções. Cada item somente poderá ser marcado como concluído após a verificação da solução e o registro de evidência no repositório. As prioridades e os identificadores correspondem aos achados descritos neste relatório.

SIGE-01 — P0

☐ Remover .env e logs dos pacotes compartilhados; revisar e substituir credenciais expostas; confirmar ausência de dados sensíveis no material distribuído.

Responsável: _____ Data: ____/____/_____

Evidência (commit, teste, print ou link):

_____

SIGE-02 — P0

☐ Revisar a restauração de backup, validar integridade referencial e testar restauração em ambiente isolado, incluindo cenário de falha.

Responsável: _____ Data: ____/____/_____

Evidência (commit, teste, print ou link):

_____

SIGE-03 — P1

☐ Evitar retorno de detalhes internos de exceções ao cliente; registrar erros de forma segura e validar mensagens de resposta.

Responsável: _____ Data: ____/____/_____

Evidência (commit, teste, print ou link):

_____

SIGE-04 — P1

☐ Desativar debug fora do desenvolvimento; restringir acesso ao banco e revisar configurações de implantação Docker.

Responsável: _____ Data: ____/____/_____

Evidência (commit, teste, print ou link):

_____

SIGE-05 — P1

☐ Reavaliar armazenamento de tokens no navegador e aplicar mitigação adequada; testar login, logout e expiração de sessão.

Responsável: _____ Data: ____/____/_____

Evidência (commit, teste, print ou link):

_____

SIGE-06 — P1

☐ Retirar chave de aplicação preenchida do arquivo de exemplo; gerar chaves próprias por ambiente e verificar histórico de exposição.

Responsável: _____ Data: ____/____/_____

Evidência (commit, teste, print ou link):

_____

SIGE-07 — P2

☐ Uniformizar versão mínima de PHP declarada e validar instalação com a versão documentada.

Responsável: _____ Data: ____/____/_____

Evidência (commit, teste, print ou link):

_____

SIGE-08 — P2

☐ Consolidar instruções de instalação e execução; testar o passo a passo em ambiente limpo.

Responsável: _____ Data: ____/____/_____

Evidência (commit, teste, print ou link):

_____

SIGE-09 — P2

☐ Ampliar testes automatizados para fluxos críticos, autorização e falhas; registrar resultado da execução.

Responsável: _____ Data: ____/____/_____

Evidência (commit, teste, print ou link):

_____

SIGE-10 — P2

☐ Revisar renderização de HTML no chatbot; sanitizar conteúdo quando necessário e testar entradas não confiáveis.

Responsável: _____ Data: ____/____/_____

Evidência (commit, teste, print ou link):

_____

SIGE-11 — P2

☐ Definir escopo do backup e documentar o que é ou não restaurado; validar recuperação dos dados necessários.

Responsável: _____ Data: ____/____/_____

Evidência (commit, teste, print ou link):

_____

SIGE-12 — P3

☐ Organizar pastas e projetos paralelos, identificar a versão oficial e remover duplicações desnecessárias com segurança.

Responsável: _____ Data: ____/____/_____

Evidência (commit, teste, print ou link):

_____

Validação final da equipe

☐ Todos os itens P0 foram corrigidos e retestados.

☐ Todos os itens P1 foram corrigidos e retestados, ou possuem justificativa formal para pendência.

☐ Os testes automatizados foram executados e os resultados registrados.

☐ O README e o repositório foram atualizados com as correções e evidências.

☐ As pendências restantes têm responsável, prioridade e prazo definidos.

Observações e pendências:

_____

_____

_____

Justificativa técnica das correções e critérios para implantação

Por que as correções são necessárias

As correções apontadas na auditoria buscam reduzir riscos de acesso indevido, exposição ou perda de informações, falhas operacionais e indisponibilidade do SIGE. Os achados relacionados à configuração, ao tratamento de dados, ao backup, à autenticação e à interface devem ser tratados conforme sua prioridade e confirmados por testes. A existência de um problema no código ou na configuração não significa, por si só, que tenha ocorrido um incidente; significa que há uma condição que precisa ser corrigida ou mitigada antes de assumir o risco de operação.

Riscos de disponibilizar o sistema sem os ajustes

Uma implantação sem a revisão dos itens de segurança pode expor credenciais, informações internas e dados armazenados; ampliar a superfície de ataque; e permitir comportamentos inesperados em determinadas entradas ou permissões. Falhas em rotinas de backup e restauração podem comprometer a integridade e a recuperação dos registros. Configurações adequadas apenas ao desenvolvimento também podem afetar a estabilidade, a confidencialidade e a capacidade de suporte do ambiente de produção.

Por que o deploy deve ser condicionado à validação

Não se recomenda autorizar o deploy em produção com dados reais enquanto houver achados críticos ou altos de segurança e integridade sem correção comprovada ou mitigação formalmente aprovada. A liberação deve depender da revisão de segredos e permissões, da configuração segura do ambiente, da execução de testes funcionais e de regressão, da verificação do backup e da documentação das evidências. Itens de menor gravidade podem ser programados, desde que seu risco residual seja avaliado e aceito pelo responsável técnico.

Condições mínimas para a liberação

Antes da implantação, a equipe deve registrar, para cada item prioritário, a alteração realizada, o responsável, a data, a evidência do teste e o resultado obtido. É necessário confirmar que não há credenciais expostas no pacote de implantação, que o ambiente está configurado para produção, que as regras de acesso funcionam

conforme os perfis e que existe procedimento de recuperação validado. A aprovação final deve ser documentada pela equipe responsável e pela gestão técnica do sistema.

Conclusão e recomendação

Diante dos riscos identificados na análise do material fornecido, a recomendação é manter o SIGE fora de produção com dados reais até a conclusão e validação dos itens impeditivos do checklist. Implantar o sistema sem essas verificações pode gerar riscos de segurança, perda de dados, indisponibilidade e necessidade de correções emergenciais. Esta recomendação é preventiva e não substitui testes em ambiente de homologação, avaliação da infraestrutura de destino ou decisão formal de aceite dos riscos residuais.